Tugas Pendahuluan
Desain dan Analisis Algoritma

Nama: Nayunda Krisna Abi Pasya

NPM: 4524210077

1. Apa yang dilakukan pada tahap desain algoritma?

Pada tahap desain algoritma, fokus utamanya adalah merancang langkah-langkah logis untuk menyelesaikan suatu masalah secara sistematis dan efisien. Kegiatan yang dilakukan meliputi:

- **Memahami Masalah:** Menganalisis input, output, dan batasan (constraints) dari masalah yang akan diselesaikan.
- **Memilih Strategi Desain:** Menentukan pendekatan yang paling sesuai, seperti *brute-force*, *divide and conquer*, *greedy*, *dynamic programming*, atau *backtracking*.
- **Merancang Langkah-langkah (Pseudocode/Flowchart):** Menuliskan urutan instruksi yang jelas dan tidak ambigu untuk menyelesaikan masalah. Representasi ini harus independen dari bahasa pemrograman manapun.
- **Menentukan Struktur Data:** Memilih struktur data yang tepat (misalnya, array, linked list, tree, graph) untuk menyimpan dan mengelola data secara efisien.

2. Apa yang dilakukan pada tahap analisis algoritma?

Tahap analisis algoritma adalah proses untuk mengevaluasi dan mengukur efisiensi dari algoritma yang telah dirancang. Tujuannya adalah untuk memprediksi sumber daya yang dibutuhkan oleh algoritma. Kegiatan utamanya adalah:

- **Analisis Kebenaran (Correctness):** Memastikan bahwa algoritma akan menghasilkan output yang benar untuk semua kemungkinan input yang valid.
- **Analisis Efisiensi:** Mengukur performa algoritma berdasarkan dua metrik utama:

- **Kompleksitas Waktu (Time Complexity):** Seberapa banyak waktu yang dibutuhkan algoritma untuk berjalan sebagai fungsi dari ukuran input.
- **Kompleksitas Ruang (Space Complexity):** Seberapa banyak memori yang digunakan algoritma sebagai fungsi dari ukuran input.

3. Komponen apa saja yang dianalisis dari sebuah algoritma?

Komponen utama yang dianalisis dari sebuah algoritma adalah:

1. **Kebenaran (Correctness):** Apakah algoritma selalu memberikan hasil yang benar dan berhenti pada semua input yang valid?
2. **Efisiensi (Efficiency):**
   - **Kompleksitas Waktu (Time Complexity):** Jumlah operasi komputasi dasar (seperti perbandingan, penjumlahan) yang dilakukan.
   - **Kompleksitas Ruang (Space Complexity):** Jumlah unit memori maksimum yang diperlukan selama eksekusi.
3. **Kesederhanaan (Simplicity):** Apakah algoritma mudah dipahami, diimplementasikan, dan dimodifikasi? Algoritma yang sederhana cenderung memiliki lebih sedikit bug.
4. **Optimalitas (Optimality):** Apakah tidak ada algoritma lain yang lebih baik (lebih efisien) untuk menyelesaikan masalah yang sama?

4. Apa perbedaan running time dan kompleksitas? Apa hubungan diantara keduanya? Bagaimana cara mengukurnya?

- **Perbedaan:**
  - **Running Time:** Adalah waktu *aktual* (dalam detik, milidetik) yang dibutuhkan sebuah program untuk dieksekusi pada mesin tertentu dengan input tertentu. Nilainya sangat

bergantung pada faktor eksternal seperti *hardware* (kecepatan prosesor), *compiler*, dan sistem operasi.

- **Kompleksitas (Waktu):** Adalah representasi *abstrak* dan teoritis tentang bagaimana jumlah operasi dalam algoritma tumbuh seiring dengan bertambahnya ukuran input. Ini bersifat independen dari mesin dan lingkungan eksekusi.

- Hubungan:

  Kompleksitas digunakan untuk memprediksi bagaimana running time akan berubah (bertambah) ketika ukuran input menjadi sangat besar. Algoritma dengan kompleksitas waktu yang lebih rendah secara umum akan memiliki running time yang lebih cepat untuk input berukuran besar.

- **Cara Mengukur:**
  - **Running Time:** Diukur secara empiris dengan menggunakan fungsi pencatat waktu (timer/stopwatch) sebelum dan sesudah eksekusi kode pada data uji.
  - **Kompleksitas:** Diukur secara analitis menggunakan **Notasi Asimtotik (Asymptotic Notation)**, terutama **Big O Notation (**O**)** yang menggambarkan batas atas (kasus terburuk/worst-case). Contoh: O(1) (konstan), O(logn) (logaritmik), O(n) (linear), O(nlogn), O(n2) (kuadratik).

5. Sebutkan algoritma sorting dan searching yang SUDAH Anda pelajari sebelumnya!

Berikut adalah beberapa contoh algoritma sorting dan searching yang umum dipelajari pada tingkat dasar dan menengah:

- **Algoritma Searching:**
  - **Linear Search:** Mencari data dengan menelusuri setiap elemen dari awal hingga akhir.
  - **Binary Search:** Mencari data pada *array yang sudah terurut* dengan membagi dua array secara berulang hingga elemen ditemukan.
- **Algoritma Sorting:**
  - **Bubble Sort:** Membandingkan dan menukar pasangan elemen yang bersebelahan

secara berulang.

- **Selection Sort:** Mencari elemen terkecil dan memindahkannya ke posisi yang benar.
- **Insertion Sort:** Membangun array akhir yang terurut satu per satu elemen.
- **Merge Sort:** Menggunakan pendekatan *divide and conquer* untuk memecah, mengurutkan, dan menggabungkan kembali array.
- **Quick Sort:** Menggunakan pendekatan *divide and conquer* dengan memilih sebuah 'pivot' untuk mempartisi array.

6. Sebutkan algoritma lain yang sudah Anda pelajari sebelumnya dan apa problem atau konteks penyelesaian algoritma tersebut?

Berikut adalah contoh algoritma lain di luar sorting/searching beserta konteks masalahnya:

- **Algoritma Dijkstra:**
  - **Problem:** Menemukan jalur terpendek dari satu titik (node) ke semua titik lain dalam sebuah graf berbobot (weighted graph) di mana bobotnya tidak negatif.
  - **Konteks:** Sistem navigasi GPS (mencari rute tercepat), perutean jaringan komputer (menemukan jalur pengiriman data paling efisien).
- **Algoritma Kruskal dan Prim:**
  - **Problem:** Mencari *Minimum Spanning Tree* (MST) dari sebuah graf berbobot terhubung. MST adalah subgraf yang menghubungkan semua titik dengan total bobot minimum.
  - **Konteks:** Desain jaringan seperti jaringan pipa, kabel listrik, atau jaringan telekomunikasi untuk menghubungkan beberapa lokasi dengan total biaya pemasangan minimum.
- **Algoritma Knapsack Problem (Dynamic Programming):**
  - **Problem:** Diberikan sekumpulan barang dengan berat dan nilai masing-masing, tentukan kombinasi barang yang dapat dimasukkan ke dalam sebuah "ransel" dengan

kapasitas berat terbatas untuk mendapatkan total nilai maksimum.

- **Konteks:** Alokasi sumber daya, manajemen investasi, dan masalah optimisasi lainnya di mana ada batasan dan tujuan untuk memaksimalkan keuntungan.